HALOBASE Case Study
Sensor Data Processing and Noise Filtering

Mustafa Oğuz Haktan
Department of Physics, Bilkent University, Ankara, Türkiye
(May 7, 2026)

DATA PROCESSING PART

I. REMOVING THE SPIKES BY MEDIAN FILTER

Before applying the main noise reduction method of Exponential Moving Average (EMA) filter, I first wanted to remove the peaks caused by the AC/HVAC disturbance. This way, EMA method would filter things out more efficiently. I used the Median filter.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import style
# Load the CSV
signal = pd.read_csv("halobase_sensor_test.csv")

raw = signal["raw_sensor_value"].values.copy()
y = raw.copy()
x = signal["time"]


# Plot the histogram
plt.figure(figsize=(10,8))
plt.hist(y,50)
plt.xlabel("Amplitude")
plt.ylabel("Amount")
plt.title("Signal Distribution")
plt.show() # Define  tresholds

uptreshold = 61
lowtreshold = 38
# Applying Median Filter for spikes
ultra_tresh = np.where((y > uptreshold) | (y < lowtreshold))[0]
filtsig = np.copy(y)

N = 5

for ii in range(0, ultra_tresh.shape[0]):
    i = ultra_tresh[ii]

    start_i = max(0, i - N)
    end_i = min(len(y), i + N + 1)

    filtsig[i] = np.median(y[start_i : end_i])
```

Listing 1: Median Filter

By plotting the data as a histogram, I spotted the lower and upper thresholds and eliminated them with the Median Filter algorithm that scans the 5 points at the neighborhood of a point and takes the median as the value of that point. Then I listed my filtered data as filtsig.

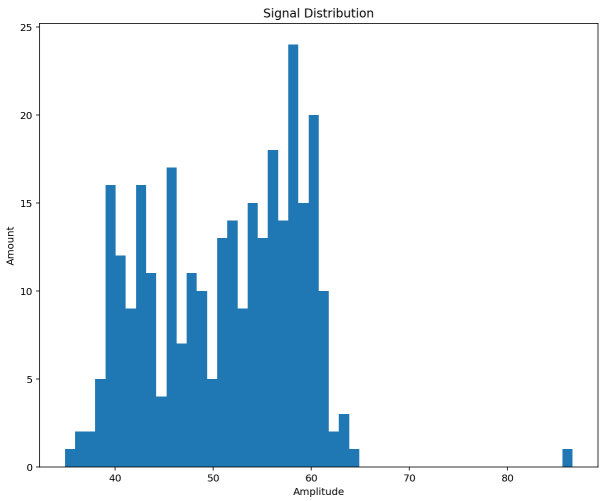

Figure 1: The histogram.

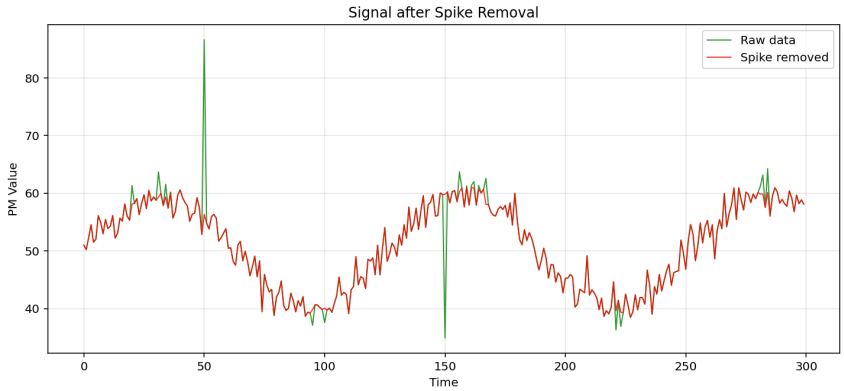

Figure 2: Spikes removed.

One can do this process in one line using modules but I wanted to show the process behind the filtering method.

II. FILTERING THE NOISE BY EMA

Using the EMA which is shown as

$$y_k = \alpha u_k + (1 - \alpha) y_{k-1}$$

to the filtsig as $\alpha = 0.2$ for a balance between the smoothness and phase difference, I got this plot:

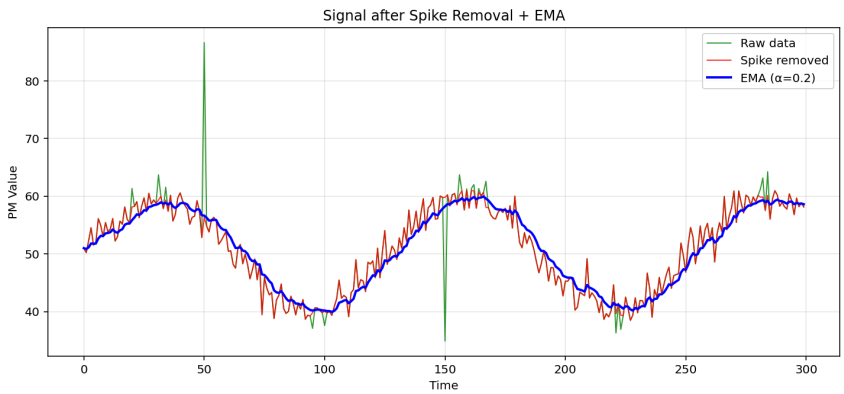

Figure 3: Spikes removed.

```
# Now we shall apply EMA to our non-spiked signal
alpha = 0.2

ema = [filtsig[0]]
for k in filtsig[1:]:
    ema.append(alpha * k + (1 - alpha) * ema[-1])

ema = np.array(ema)
```

Listing 2: Median Filter

I preferred EMA because it was easy to apply and since the signal has long periods its fairly accurate. It also only holds an alpha value and the previous point's value, which is good on RAM.

LATENCY, CPU AND RAM COMMENT

- The latency is very low because Median filter applies at most 5 sample delays since N=5. For the EMA, time constant τ is given as $\frac{1}{\alpha} = 5$ for our case. Overall, latency is really low and acceptable for this case.
- Median filter only sorts 5 numbers at a time which takes tiny CPU power. EMA method only memorizes two values, alpha and previous point value y_{k-1}. So the overall CPU and RAM power needed is very low. I believe this can run on even an Ardunio UNO.
- For a quite room with little noise, a bigger α value would be better since there is not much noise. This way, the measurement in lag can be compensated for. For a busy environment, there would be much more noise. Therefore, a lower α value that corresponds to more noise cancelling would be better. The tradeoff would be more lag but more accurate and safe measurements. Plots for low and high α are provided as such:

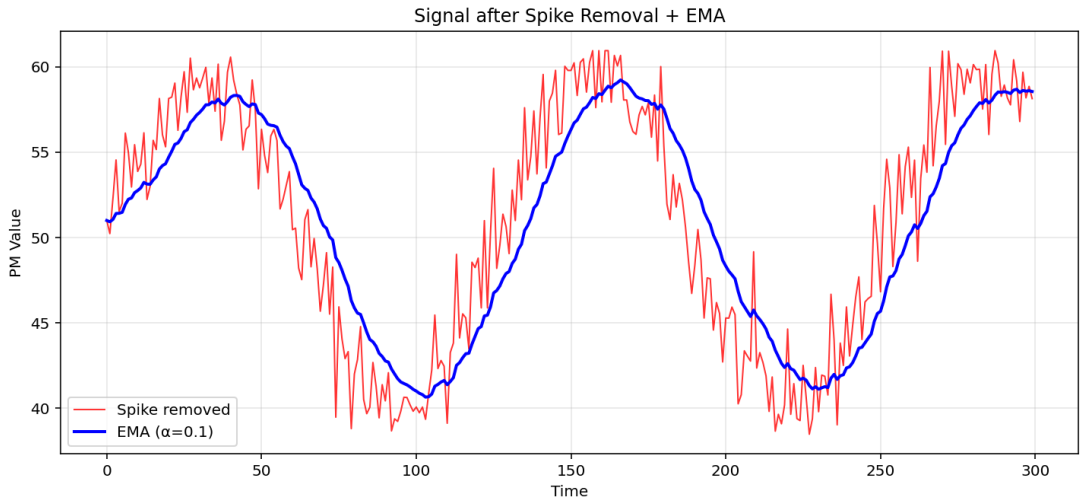

Figure 4: For $\alpha = 0.1$. A delay in signals but much reduces noise can be seen. Better for busy enviroments.

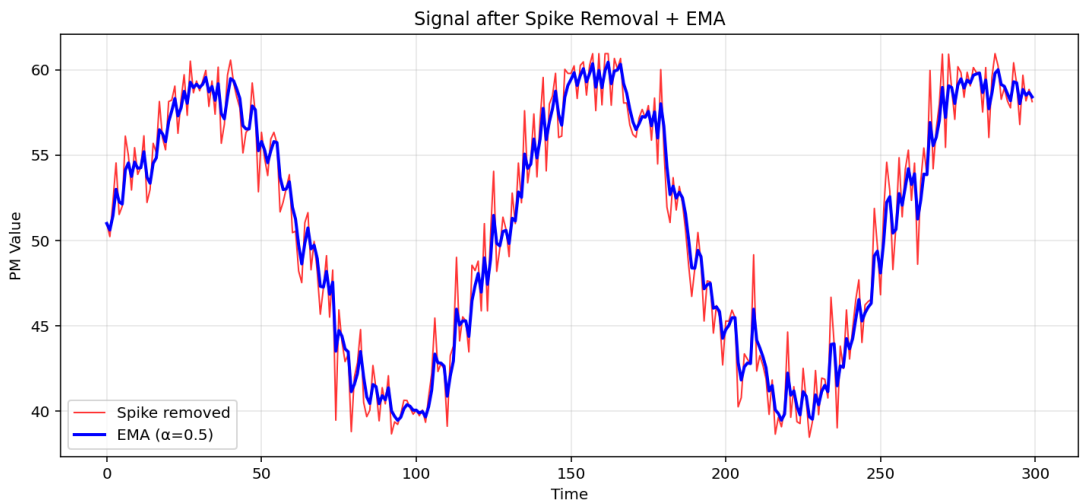

Figure 5: For $\alpha = 0.5$. The signals have no delay between them. However the reducing in noise is much worse. Better for more stable environments.

A script to change the alpha value in real time according to the measurements taken can be written.